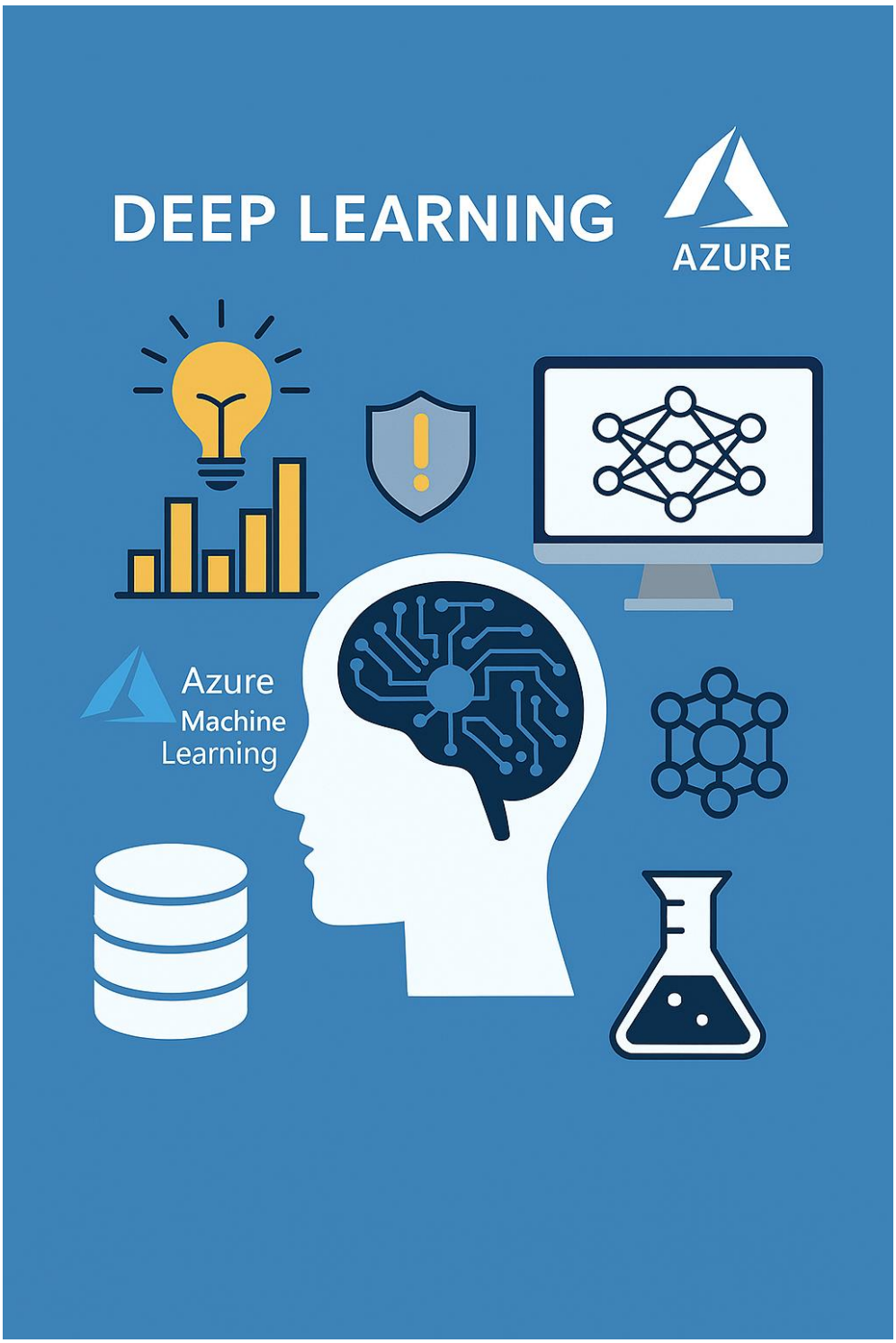
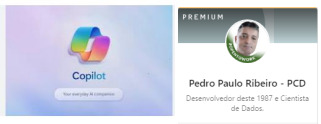
Voluntary Contribution

https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.





Voluntary Contribution

https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

Deep Learning on Azure: From Theory to Practice

Foreword

- Book presentation
- Why Deep Learning on Azure?
- Who is this book for?

Chapter 1: Introduction to Deep Learning

- What is deep learning?
- Artificial neural networks: an overview
- Historical Evolution of Deep Learning
- Real-world applications: computer vision, NLP, recommendation

Chapter 2 : Microsoft Azure Overview

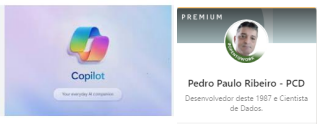
- What is Azure?
- Main services for AI and Machine Learning
- Comparison with other platforms (AWS, GCP)
- Security, scalability and compliance

Chapter 3: AI Services on Azure

- Azure Machine Learning (AML)
- Azure Cognitive Services
- Azure Databricks
- Azure Synapse Analytics
- Azure OpenAI Service

Chapter 4: Preparing the Environment

- Creating an Azure account
- Configuring your Azure Machine Learning workspace
- Creating notebooks and pipelines
- Managing resources: VMs, containers, and clusters



Voluntary Contribution

https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

Chapter 5: Deep Learning Modeling Fundamentals

- Types of networks: CNN, RNN, LSTM, GANs
- Supported frameworks: TensorFlow, PyTorch, Scikit-learn
- Data Preprocessing with Azure Data Factory
- Local training vs. distributed training

Chapter 6: Model Training on Azure

- Using AML for Automated Training
- Experiments and Tracking with MLFlow
- Hyperparameters and cross-validation
- GPU Usage and Scalability with Batch AI

Chapter 7: Model Deployment

- Creating REST Endpoints with AML
- Deploy in containers (ACI, AKS)
- Monitoring and logging with Application Insights
- Model update and versioning

Chapter 8: Cognitive Services with Deep Learning

- Computer vision: OCR, object detection
- Natural language processing: translation, sentiment analysis
- Facial recognition and biometrics
- Customizing pre-trained models

Chapter 9: Governance and Ethics in AI

- Fairness, explainability and transparency
- Audit Tools in Azure
- LGPD and GDPR: legal compliance
- Responsible AI and Best Practices

Voluntary Contribution

https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

Chapter 10: Real-World Use Cases

- Medical diagnosis with images
- Retail demand forecast
- Financial fraud detection
- Intelligent Chatbots with Azure OpenAI

Chapter 11: Integration with Power BI and Apps

- Connecting models to Power BI
- Web applications with Flask and Streamlit
- APIs and microservices with Azure Functions
- Automation with Logic Apps

Chapter 12: Certifications and Careers

- Microsoft Certifications in AI and Azure
- Professional profiles: Data Scientist, ML Engineer
- Interview and Portfolio Tips
- AI Communities and Events

Chapter 1: Introduction to Deep Learning

1.1 What is Deep Learning?

Deep Learning is a subfield of Machine Learning that uses multi-layer artificial neural networks to learn complex data representations. Unlike traditional algorithms, deep learning can automatically extract relevant features from raw data, such as images, text, or audio.

Main features:

- Hierarchical representation learning
- Generalization capability in large volumes of data
- Intensive use of computing resources (GPU/TPU)

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

1.2 Artificial Neural Networks

Neural networks are composed of artificial neurons organized in layers:

- **Input layer** : receives the raw data
- **Hidden layers** : perform non-linear transformations
- **Output layer** : generates the prediction or classification

Common types of networks:

Network Type	Main Application
CNN (Convolutional Neural Network)	Computer vision
RNN (Recurrent Neural Network)	Time series and text
LSTM (Long Short-Term Memory)	Machine translation, chatbots
GAN (Generative Adversarial Network)	Generation of synthetic images and data

1.3 Historical Evolution of Deep Learning

- **1950s–1980s** : first models of artificial neurons (Perceptron)
- **1990s** : Multilayer networks and backpropagation
- **2006** : Rebirth of deep learning with deep networks
- **2012** : AlexNet wins computer vision competition (ImageNet)
- **2020s** : Foundational models such as GPT, BERT, DALL·E and CLIP

1.4 Comparison with Traditional Machine Learning

Aspect	Traditional Machine Learning	Deep Learning
Feature extraction	Manual	Automatic
Data volume	Moderate	Massive
Training time	Fast	Slow
Interpretability	High	Low
Applications	Tabular, statistics	Images, text, sound

1.5 Real Applications

- **Health** : imaging diagnosis, disease prediction
- **Finance** : fraud detection, risk analysis
- **Retail** : product recommendations, demand forecasting
- **Industry** : predictive maintenance, quality control
- **Entertainment** : content generation, personalization

Deep Learning on AZURE.

1.6 Why use Deep Learning on Azure?

Microsoft Azure provides a robust and scalable infrastructure for training, testing, and deploying deep learning models with:

- GPUs and TPUs on demand
- Managed services like Azure Machine Learning
- Integration with popular frameworks (TensorFlow, PyTorch)
- Monitoring, versioning and security tools

Chapter 2 : Microsoft Azure Overview

2.1 What is Azure?

Microsoft Azure is a cloud computing platform that offers over 200 products and services for building, testing, deploying, and managing intelligent applications and solutions. It enables developers and data scientists to leverage scalable infrastructure, managed services, and integrated tools to accelerate the development of AI solutions.

Main features:

- Global infrastructure with data centers in more than 60 regions
- Support for multiple operating systems and languages
- Corporate security and compliance with international standards
- Integration with Microsoft tools such as Power BI, GitHub and Visual Studio

2.2 Azure for Artificial Intelligence

Azure offers a complete ecosystem for AI, including:

Azure Machine Learning (AML)

- Managed service for model training, validation, and deployment
- Support for frameworks such as TensorFlow, PyTorch, and Scikit-learn
- Automated pipelines, experiment tracking, and model versioning

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

Azure Cognitive Services

- Pre-trained APIs for vision, speech, language, and decision-making
- Examples: OCR, machine translation, sentiment analysis, facial recognition

Azure OpenAI Service

- Access to GPT, Codex, and DALL·E models with enterprise security
- Ideal for generating text, code, images, and smart assistants

Azure Databricks

- Collaborative environment based on Apache Spark
- Ideal for large-scale data processing and integration with notebooks

Azure Synapse Analytics

- Integrated analytics platform for big data and data warehousing
- Supports SQL queries, Spark, and Power BI integration

2.3 Security, Scalability and Compliance

Azure is recognized for its robust security and compliance:

- **Security** : data encryption at rest and in transit, multi-factor authentication, firewalls
- **Scalability** : automatic resource provisioning, load balancing, GPU clusters
- **Compliance** : compliance with ISO, SOC, GDPR, LGPD, HIPAA

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

2.4 Comparison with Other Platforms

Platform	Strengths	Limitations
Azure	Microsoft integration, security	Initial learning curve
AWS	Wide range of services, maturity	Configuration complexity
Google Cloud	Strong in AI and data	Less integration with corporate environments

2.5 When to Use Azure for Deep Learning?

Azure is ideal for:

- Corporate projects that require security and compliance
- Integration with Microsoft tools (Excel, Power BI, Teams)
- Distributed training using GPU/TPU
- Deployment of scale models with monitoring

2.6 Practical Experience

Over the next few chapters, you will learn how to:

- Create a Machine Learning workspace
- Train models with real data
- Deploy models as scalable APIs
- Monitor and update models in production

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

Chapter 3: AI Services on Azure

3.1 AI Services Overview

Azure offers a robust and integrated ecosystem of services to support the entire lifecycle of artificial intelligence projects, from data ingestion to model deployment in production. Key services include:

- **Azure Machine Learning (AML)**
- **Azure Cognitive Services**
- **Azure OpenAI Service**
- **Azure Databricks**
- **Azure Synapse Analytics**

3.2 Azure Machine Learning (AML)

Azure Machine Learning is the backbone for machine learning and deep learning projects in the Microsoft cloud.

Key Features:

- **Workspaces** : isolated environments for organizing experiments, models, and pipelines
- **Integrated Notebooks** : Jupyter support with libraries like TensorFlow, PyTorch, Scikit-learn
- **Automated Training** : AutoML to Find the Best Models and Hyperparameters
- **ML Pipelines** : Orchestration of steps such as preprocessing, training, and validation
- **Deployment** : REST endpoints, Docker containers, AKS (Kubernetes)

Example of use:
Python
`from azureml.core import Workspace, Experiment`
`ws = Workspace.from_config()`
`experiment = Experiment(workspace=ws, name='my-experiment-dl')`

3.3 Azure Cognitive Services

A set of pre-trained APIs that allow you to add intelligence to applications without having to train models from scratch.

Categories:

- **Computer Vision** : OCR, object detection, image analysis
- **Language** : sentiment analysis, translation, entity extraction
- **Speech** : text-to-speech and speech-to-text conversion, speaker recognition
- **Decision** : customization, anomaly detection

Example:
http
`POST https://<endpoint>/vision/v3.2/analyze`
`Headers: Ocp-Apim-Subscription-Key: <your-key>`
`Body: { "url": "https://image.jpg" }`

3.4 Azure OpenAI Service

Provides access to OpenAI models with enterprise-grade security and native integration with Azure.

Available models:

- **GPT-4** : text generation, summarization, translation
- **Codex** : code generation
- **DALL·E** : generating images from text
- **Whisper** : Audio Transcription

Use cases:

- Intelligent chatbots
- Content generation
- Programming assistants
- Document analysis

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

3.5 Azure Databricks

Apache Spark-based data analytics platform ideal for preprocessing large volumes of data before model training.

Resources:

- Collaborative notebooks
- Integration with AML and Synapse
- MLflow support for experiment tracking
- Automatic scalability

3.6 Azure Synapse Analytics

Unified data analytics platform that combines data warehouse with big data.

AI Applications:

- Data preparation for models
- SQL and Spark Queries
- Integration with Power BI and AML

3.7 Integration between Services

Azure's biggest advantage is the seamless integration between its services:

- Data from Synapse → preprocessed in Databricks → trained in AML → deployed via AKS → visualized in Power BI
- Cognitive Services and OpenAI can be used as auxiliary APIs in AML pipelines

3.8 When to Use Each Service?

Objective	Ideal Service

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

Train models from scratch	Azure Machine Learning
Use ready-made templates	Azure Cognitive Services
Text or code generation	Azure OpenAI Service
Preprocess large datasets	Azure Databricks
Analyze and integrate data	Azure Synapse Analytics

Chapter 4: Preparing the Environment

4.1 Creating an Azure Account

First of all, you need a Microsoft Azure account. You can create one for free at azure.microsoft.com and get started with free trial credits.

Steps:

1. Visit the Azure website
2. Click “Start for free”
3. Create a Microsoft account or use an existing one
4. Verify identity and payment method
5. Access the portal: portal.azure.com

4.2 Creating a Workspace in Azure Machine Learning

The workspace is the central environment for managing all your Machine Learning resources.

Steps:

- In the Azure portal, click “Create resource”
- Select “Machine Learning”
- Configure:
  - Workspace name
  - Resource group
  - Location (region)
  - Storage type and container

Example with Python SDK:
Python
`from azureml.core import Workspace`
`ws = Workspace.create(name='my_workspace',`
`subscription_id='xxxx-xxxx-xxxx',`
`resource_group='my_group',`
`location='brazilsouth',`
`create_resource_group=True,`
`exist_ok=True)`

4.3 Working with Notebooks

Azure ML Studio offers built-in Jupyter notebooks with support for Python, R, and other languages.

Resources:

- GPU-capable kernel
- Direct access to workspace and datasets
- GitHub Integration

Tip:

You can import existing notebooks or create from scratch. Use libraries like `tensorflow`, `torch`, `sklearn`, `matplotlib`, and `pandas`.

4.4 Managing Computing Resources

You can create and manage compute clusters to train models.

Types:

- **Compute Instance** : ideal for interactive development
- **Compute Cluster** : for large-scale training
- **Inference Cluster** : for model deployment

Example:

Python

```
from azureml.core.compute import AmlCompute, ComputeTarget

compute_config = AmlCompute.provisioning_configuration(vm_size='STANDARD_NC6',
max_nodes=4)
cluster = ComputeTarget.create(ws, 'gpu-cluster', compute_config)
```

4.5 Managing Data

Azure ML allows you to register, version, and share datasets.

Data sources:

- Azure Blob Storage
- Azure Data Lake
- Local files
- Public URLs

Example:

Python

```
from azureml.core.dataset import Dataset

dataset = Dataset.File.from_files(path=['https://dados.com/imagens.zip'])
dataset.register(workspace=ws, name='training_images')
```

4.6 Automating with Pipelines

You can create pipelines to automate tasks like preprocessing, training, and validation.

Components:

- Stages (steps)
- Inputs and outputs
- Dependencies (conda, docker)

Example:
Python
`from azureml.pipeline.core import Pipeline`
`pipeline = Pipeline(workspace=ws, steps=[step_preprocess, step_train])`
`pipeline.run()`

4.7 Security and Access Control

Azure offers granular permissions control:

- RBAC (Role-Based Access Control)
- Managed identity
- Logging and Auditing with Azure Monitor

4.8 Environment Checklist

✅ Account created ✅ Workspace configured ✅ GPU cluster provisioned ✅ Notebook ready ✅ Dataset registered ✅ Pipeline defined

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

Chapter 5: Deep Learning Modeling Fundamentals

5.1 Types of Neural Networks

Deep Learning is based on deep neural network architectures. The main ones are:

Network Type	Common Applications
CNN (Convolutional Neural Network)	Computer vision, image classification
RNN (Recurrent Neural Network)	Time series, sequential text
LSTM (Long Short-Term Memory)	Machine translation, chatbots
GAN (Generative Adversarial Network)	Image generation, synthetic data

Each architecture has specific characteristics that make it ideal for different types of data and problems.

5.2 Supported Frameworks on Azure

Azure natively supports the main Deep Learning frameworks:

- **TensorFlow** : ideal for convolutional networks and sequential models
- **PyTorch** : Flexible and popular among researchers
- **Keras** : High-level interface to TensorFlow
- **Scikit-learn** : useful for traditional ML tasks and preprocessing

You can use these frameworks in Azure Machine Learning notebooks or on Azure Databricks clusters.

5.3 Data Preprocessing with Azure Data Factory

Before training models, it is essential to prepare the data:

- Cleaning and standardization
- Coding of categorical variables
- Division into training, validation and test sets
- Class balancing

Azure **Data Factory** lets you build ETL pipelines to transform data at scale, integrating with sources like SQL Server, Blob Storage, and external APIs.

Deep Learning on AZURE.

5.4 Local vs. Distributed Training

Local Training:

- Ideal for rapid prototyping and testing
- Uses limited resources (local CPU/GPU)

Distributed Training:

- Use GPU clusters on Azure
- Enables data and model parallelism
- Support for libraries like **TorchDistributor** on Azure Databricks

Example with PyTorch on Databricks:
Python
`from torch.utils.data import DataLoader`
`from torchvision import datasets, transforms`
`train_loader = DataLoader(`
`datasets.MNIST('./data', train=True, download=True,`
`transform=transforms.ToTensor()),`
`batch_size=64, shuffle=True)`

5.5 Good Modeling Practices

- Use cross-validation to avoid overfitting
- Monitor metrics such as loss, accuracy, precision, and recall
- Save checkpoints during training
- Use callbacks for early stopping and learning rate adjustment

5.6 Integration with Azure Machine Learning

You can register trained models, track experiments, and deploy directly via AML.

Tracking example:
Python
`from azureml.core import Experiment`
`experiment = Experiment(workspace=ws, name='experimento_dl')`
`run = experiment.start_logging()`
`run.log('accuracy', 0.92)`
`run.complete()`

5.7 Practical Exercise

Objective : Train a CNN to classify digit images (MNIST) using PyTorch on Azure Databricks.

Steps :

1. Create notebook in Databricks
2. Import MNIST dataset
3. Define CNN architecture
4. Train model with TorchDistributor
5. Evaluate and save model

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

🚀 Chapter 6: Model Training on Azure

🧠 6.1 Training Strategies

There are different ways to train models on Azure, depending on the size of your data, the complexity of your model, and the resources available:

- **On-site training** : useful for rapid prototyping and testing
- **Compute instance training** : ideal for interactive notebooks
- **Distributed cluster training** : recommended for large data volumes and complex models
- **AutoML** : for automating the modeling process and algorithm selection

🧮 6.2 Using Azure Machine Learning for Training

Main steps:

1. **Creation of the experiment**
2. **Training script definition**
3. **Environment setup (conda/docker)**
4. **Computing resource selection**
5. **Execution and tracking**

Example with PyTorch:

Python

```
from azureml.core import ScriptRunConfig, Environment

env = Environment.from_conda_specification(name='pytorch-env',
file_path='env.yml')

src = ScriptRunConfig(source_directory='.',
script='train.py',
compute_target='gpu-cluster',
environment=env)

run = experiment.submit(src)
run.wait_for_completion(show_output=True)
```

6.3 Monitoring with MLFlow and Azure ML

During training, it is essential to track metrics such as:

- Loss
- Accuracy
- Precision and Recall
- Execution time
- GPU/CPU Usage

You can use:

- **MLFlow** : Integrated with Azure Databricks
- **Run logs** : in Azure ML Studio
- **Application Insights** : for production monitoring

6.4 Hyperparameter Tuning

Azure ML allows you to perform automatic hyperparameter scans with:

- **Grid Search**
- **Random Search**
- **Bayesian Optimization**

Example:

Python

```
from azureml.train.hyperdrive import GridParameterSampling, HyperDriveConfig

param_sampling = GridParameterSampling({
'learning_rate': [0.01, 0.1],
'batch_size': [32, 64]
})

hd_config = HyperDriveConfig(run_config=src,
hyperparameter_sampling=param_sampling,
max_total_runs=4,
primary_metric_name='accuracy',
primary_metric_goal='MAXIMIZE')
```

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

⚡ 6.5 Using GPU and Distributed Training

Azure offers GPU machines (e.g. NC6, ND40) and distributed training support with:

- **Horovod**
- **PyTorch DDP**
- **TensorFlow MirroredStrategy**

Benefits:

- Reduction in training time
- Horizontal scalability
- Multi-node and multi-GPU support

🧪 6.6 Practical Exercise

Objective : Train a CNN to classify fruit images using PyTorch on Azure ML.

Steps :

1. Create dataset with fruit images
2. Define CNN architecture
3. Configure GPU environment and cluster
4. Run training with metrics tracking
5. Save and register the model

☑️ 6.7 Training Checklist

- [x] Training script defined
- [x] Configured environment
- [x] Provisioned compute cluster
- [x] Experiment created
- [x] Tracked metrics
- [x] Model saved and registered

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

Chapter 7: Model Deployment

7.1 What is Deployment?

Deploying a model means making it available for real-time use, usually through a REST API. This allows web and mobile applications, or enterprise systems, to make predictions based on new data.

Types of deployment:

- **Online (real time)** : immediate response via API
- **Batch** : Processing large volumes of data periodically

7.2 Preparing the Model for Deployment

Before deploying, it is necessary to:

- Save the trained model (e.g. `.pkl` , `.pt` , `.h5`)
- Register the model in Azure Machine Learning
- Set the runtime environment (dependencies, Python version)
- Create an input script (`score.py`) that defines how the model receives data and returns predictions

Example of registration:

Python

```
from azureml.core.model import Model

model = Model.register(workspace=ws,
model_path='outputs/model.pkl',
model_name='modelo_frutas')
```

7.3 Creating REST Endpoints

You can deploy the model as a web service using:

- **Azure Container Instances (ACI)** : Ideal for testing and low scale
- **Azure Kubernetes Service (AKS)** : Recommended for production and high availability

Example with ACI:

Python

```
from azureml.core.webservice import AciWebservice, Webservice

deployment_config = AciWebservice.deploy_configuration(cpu_cores=1, memory_gb=1)
service = Model.deploy(workspace=ws,
name='servico-frutas',
models=[model],
inference_config=inference_config,
deployment_config=deployment_config)
service.wait_for_deployment(show_output=True)
```

🔍 7.4 Testing the Endpoint

After deployment, you can test the service with an HTTP request:

Python

```
import requests
import json

url = service.scoring_uri
headers = {'Content-Type': 'application/json'}

data = json.dumps({'data': [[5.1, 3.5, 1.4, 0.2]]})
response = requests.post(url, data=data, headers=headers)
print(response.json())
```

📊 7.5 Monitoring and Logging

Azure offers tools to monitor model performance in production:

- **Application Insights** : Call tracking, response time, errors
- **Azure Monitor** : Infrastructure Metrics
- **Model Drift Detection** : alerts you when the model starts to lose accuracy

🔄 7.6 Update and Versioning

You can:

- Deploy new model versions without interrupting service
- Keep multiple models active (e.g. A/B testing)
- Roll back to previous versions in case of failures

Example:
Python
`model_v2 = Model.register(workspace=ws,`
`model_path='outputs/model_v2.pkl',`
`model_name='model_fruits',`
`version=2)`

🧪 7.7 Practical Exercise

Objective : Deploy a fruit classification model as a REST API.

Steps :

1. Register the trained model
2. Create `score.py` with inference logic
3. Set environment with `inference_config`
4. Deploy with ACI
5. Test with real data

☑️ 7.8 Implementation Checklist

- [x] Model saved and registered
- [x] Inference script created
- [x] Defined environment
- [x] Endpoint created and tested
- [x] Monitoring enabled
- [x] Versioning configured

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

Chapter 8: Cognitive Services with Deep Learning

8.1 What are Azure Cognitive Services?

Azure **Cognitive Services** is a set of APIs and SDKs based on Microsoft-trained Deep Learning models. They enable developers to add intelligence to applications with just a few lines of code.

Main categories:

- Vision
- Language
- He speaks
- Decision
- Search (Bing Search APIs)

8.2 Computer Vision

Allows you to analyze images and videos with deep vision models.

Features:

- Object detection
- OCR (reading text from images)
- Facial analysis (emotions, age, gender)
- Automatic image description

Example of use:
http
POST https://<endpoint>/vision/v3.2/analyze
Headers: Ocp-Apim-Subscription-Key: <your-key>
Body: { "url": "https://example.com/image.jpg" }

Deep Learning on AZURE.

8.3 Natural Language Processing (NLP)

It allows you to understand and generate human language with deep language models.

Features:

- Sentiment analysis
- Named Entity Extraction
- Automatic translation
- Grammatical correction
- Text classification

Example with Python:

Python

```
import requests

url = "https://<endpoint>/text/analytics/v3.1/sentiment"
headers = {"Ocp-Apim-Subscription-Key": "<your-key>"}
data = {"documents": [{"id": "1", "language": "pt", "text": "I'm very happy with the service!"}]}

response = requests.post(url, headers=headers, json=data)
print(response.json())
```

8.4 Speech Recognition and Synthesis

It transforms speech to text and vice versa with high accuracy.

Features:

- Speech-to-Text (STT)
- Text to Speech (TTS)
- Speaker recognition
- Real-time speech translation

Applications:

- Virtual assistants
- Meeting transcription

- Automated content reading

8.5 Decision Services

Models trained to help with intelligent decision making.

Examples:

- **Personalizer** : Real-time personalized recommendations
- **Anomaly Detector** : anomaly detection in time series
- **Content Moderator** : Moderation of offensive content

8.6 Template Customization

You can train your own models based on Cognitive Services:

- **Custom Vision** : Train an image classification model with your own data
- **Custom Translator** : customize translation templates
- **Language Studio** : Build text classifiers and entity extractors

Example with Custom Vision:

1. Upload labeled images
2. Train the model with just a few clicks
3. Export as REST API or ONNX model

8.7 Integration with Azure Machine Learning

You can use Cognitive Services as part of ML pipelines:

- Preprocess data with OCR or sentiment analysis
- Enrich datasets with extracted entities
- Use results as features for custom models

🧪 8.8 Practical Exercise

Objective : Create an application that analyzes the sentiment of customer reviews.

Steps :

1. Create a Text Analytics resource in the Azure portal
2. Get key and endpoint
3. Send texts via API
4. Display results in a dashboard with Power BI or Streamlit

☑️ 8.9 Cognitive Services Checklist

- [x] Resource created in the Azure portal
- [x] Obtained key and endpoint
- [x] API tested successfully
- [x] Integration with application completed
- [x] Interpreted and visualized results

🔐 Chapter 9: Governance and Ethics in AI

🧭 9.1 What is AI Governance?

AI governance is the set of policies, processes, and tools that ensure that artificial intelligence systems are developed and used responsibly, transparently, and in compliance with legal requirements.

Objectives:

- Avoid bias and discrimination
- Protect data and privacy

- Ensure security and reliability
- Promote transparency and explainability
- Comply with legal and regulatory standards

9.2 Responsible AI on Azure

Microsoft promotes responsible AI practices based on six principles:

1. **Fairness** : Avoid discrimination and algorithmic bias
2. **Reliability and security** : protect against failures and attacks
3. **Privacy and data security** : respect users' rights
4. **Inclusion** : Making AI accessible to everyone
5. **Transparency** : Explain how models work
6. **Accountability** : defining who is responsible for automated decisions

9.3 Azure Governance Tools

Azure Policy

- Defines rules for using AI resources
- Prevents insecure or non-compliant configurations
- You can apply specific policies to services like Azure Machine Learning and Azure OpenAI.

Microsoft Enter ID for AI Agents

- Tracks and manages AI agents built on Azure
- Allows access control and centralized auditing

Monitoring and Auditing

- Application Insights and Azure Monitor to track usage and performance
- Model drift detection and anomalous behavior alerts

⚖️ 9.4 Legal Compliance: LGPD, GDPR and Others

Azure supports compliance with laws such as:

- **LGPD (Brazil)** : personal data protection
- **GDPR (Europe)** : transparency and consent
- **HIPAA (USA)** : Health Privacy

You can configure landing zones with curated policies for AI workloads that meet these requirements.

🔍 9.5 Explainability and Transparency

Deep Learning models are complex and often considered "black boxes." Azure offers:

- **Interpretability with SHAP and LIME**
- **Feature importance visualization**
- **Detailed logs of automated decisions**

⚠️ 9.6 Ethical and Socioeconomic Risks

According to experts, the main risks include:

- Algorithmic bias and social exclusion
- Job losses due to automation
- Misuse of AI for manipulation or surveillance
- Lack of accountability for automated decisions

🧪 9.7 Practical Exercise

Objective : Create a governance policy for an AI project on Azure.

Steps :

1. Define ethical principles of the project
2. Create policies with Azure Policy
3. Configure monitoring with Application Insights
4. Document decisions and justifications
5. Validate compliance with LGPD

☑ 9.8 Governance Checklist

- [x] Defined ethical principles
- [x] Policies applied with Azure Policy
- [x] Monitoring enabled
- [x] Documented explainability
- [x] Validated legal compliance

☑ Chapter 10: Real-World Use Cases

🏥 10.1 Medical Diagnosis with Images

Challenge:

Detect diseases such as cancer, pneumonia or fractures through imaging tests (X-rays, CT scans).

Solution:

- CNN training with medical images
- Using Azure Machine Learning with GPU clusters
- Deployment via AKS for hospitals to access via API
- Monitoring with Application Insights

Benefits:

- Faster and more accurate diagnosis
- Reduction of human errors
- Clinical decision support

10.2 Retail Demand Forecasting

Challenge:

Anticipate demand for products in different regions and periods.

Solution:

- LSTM models trained with sales time series
- Integration with Azure Synapse for data ingestion
- Visualization in Power BI
- Continuous update with automated pipelines

Benefits:

- Inventory optimization
- Loss reduction
- Increased logistics efficiency

10.3 Financial Fraud Detection

Challenge:

Identify suspicious transactions in real time.

Solution:

- Training classification models with historical data
- Using Azure Databricks for Preprocessing
- Deployment as a REST service with low latency
- Integrated alerts with Azure Logic Apps

Benefits:

- Reduction of financial losses
- Increased customer confidence
- Compliance with regulatory standards

🤖 10.4 Intelligent Chatbots with Azure OpenAI

Challenge:

Create virtual assistants that understand natural language and provide helpful answers.

Solution:

- Using Azure OpenAI with GPT models
- Integration with Azure Bot Service and Teams
- Personalization with internal data
- Interaction monitoring and feedback

Benefits:

- 24/7 support
- Load reduction in call centers
- Improved user experience

🏭 10.5 Predictive Maintenance in Industry

Challenge:

Avoid failures in critical machines and equipment.

Solution:

- Regression and classification models with IoT sensors
- Data Ingestion with Azure IoT Hub
- Azure ML Training
- Automatic alerts with Azure Functions

Benefits:

- Reduction of unplanned downtime
- Savings with corrective maintenance
- Longer equipment lifespan

🧪 10.6 Practical Exercise

Objective : Choose one of the cases above and create a pilot project on Azure.

Steps :

1. Define the problem and available data
2. Choosing the appropriate model architecture
3. Configure environment and resources in Azure
4. Train and validate the model
5. Deploy and monitor in production

☑️ 10.7 Real Project Checklist

- [x] Problem defined
- [x] Data collected and prepared
- [x] Trained and validated model
- [x] Implementation completed
- [x] Active monitoring
- [x] Evaluated results

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

Chapter 11: Integration with Power BI and Applications

11.1 Connecting Models to Power BI

Power BI lets you create interactive dashboards with data and predictions generated by AI models.

Forms of integration:

- **REST API** : Power BI consumes predictions from an Azure ML endpoint
- **Azure Synapse** : Connect data and model results directly to Power BI
- **Dataflows** : Automate data ingestion and transformation with built-in AI

Example:

1. Deploy the model as a REST endpoint
2. Create a custom function in Power BI using `Web.Contents`
3. Submit data and view real-time forecasts

11.2 Web Applications with Flask and Streamlit

You can create simple web interfaces for users to interact with your models.

With Flask:
• Create an API or web app with routes for uploading data and displaying results
• Integrate with the Azure ML endpoint
Python
`@app.route('/prever', methods=['POST'])`
`def predict():`
`data = request.json`
`response = requests.post(endpoint, json=data, headers=headers)`
`return response.json()`
With Streamlit:
• Ideal for prototypes and interactive dashboards
• Support for uploading files, charts, and widgets
Python
`import streamlit as st`
`st.title("Image Classifier")`
`image = st.file_uploader("Upload an image")`

11.3 APIs and Microservices with Azure Functions

You can turn your model into a scalable microservice with **Azure Functions** .

Advantages:

- Automatic scalability
- Low cost with billing per execution
- Integration with Logic Apps, Event Grid, and Power Automate

Example:

1. Create an Azure Function in Python
2. Call the model endpoint
3. Return the response as JSON

⚙ 11.4 Automation with Logic Apps

Allows you to create automated workflows with AI.

Examples:

- Send emails with forecasts
- Store results in a database
- Trigger alerts based on ratings

Flow example:

1. Trigger: New file in Blob Storage
2. Action: Send image to computer vision model
3. Action: Save result to SharePoint

📱 11.5 Integration with Mobile Applications

You can use:

- **Power Apps** : for creating mobile apps without code
- **Xamarin/MAUI** : For native apps with AI API calls
- **React Native** : with REST endpoint integration

🧪 11.6 Practical Exercise

Objective : Create a Power BI dashboard that consumes predictions from a text classification model.

Steps :

1. Deploy model as REST API
2. Create custom function in Power BI
3. Send texts and display results
4. Automatically update with Power Automate

☑️ 11.7 Integration Checklist

- [x] Model deployed as endpoint
- [x] API tested successfully
- [x] Application connected to the model
- [x] Visualization or automation created
- [x] Flow validated with real data

🎓 Chapter 12: Certifications and Careers

🎡 12.1 Why Invest in Certifications?

Certifications are official recognition of your technical skills. They help you:

- Validate your knowledge
- Increase your employability
- Open doors for projects and consultancies
- Demonstrate mastery of specific Azure tools

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

12.2 Microsoft Certifications in AI and Azure

Main certifications:

Certification	Focus	Level
Azure AI Fundamentals (AI-900)	Fundamentals of AI and Cognitive Services	Beginner
Azure Data Scientist Associate (DP-100)	Modeling, Training, and Deployment with Azure ML	Intermediary
Azure Solutions Architect Expert (AZ-305)	Cloud solutions architecture	Advanced
Azure Developer Associate (AZ-204)	Application development with AI integration	Intermediary

Tip:

Use Microsoft Learn to study for free with interactive trails and hands-on labs.

12.3 Professional Profiles in AI

Main profiles:

- **Data Scientist** : analyzes data, builds models, interprets results
- **Machine Learning Engineer** : develops pipelines, optimizes models, manages production
- **AI Solutions Architect** : designs scalable and secure systems
- **AI Developer** : Integrates models into web, mobile, and API applications

Valued skills:

- Programming (Python, SQL)
- Knowledge of frameworks (TensorFlow, PyTorch)
- Experience with Azure ML, Databricks, Synapse
- Soft skills: communication, critical thinking, ethics

12.4 Building an AI Portfolio

A good portfolio shows your ability to solve real problems.

Include:

- Projects with public data (e.g. Kaggle, UCI)
- Applications with Azure ML and Cognitive Services
- Interactive dashboards with Power BI
- Clear documentation and clean code on GitHub

12.5 Interview Tips

- Learn the fundamentals of AI and Deep Learning
- Prepare simple explanations for complex models
- Show how you deal with real data and practical problems
- Demonstrate knowledge of governance and responsible AI
- Practice with technical questions and code challenges

12.6 Communities and Events

Participating in communities helps you learn, teach, and connect.

Where to get involved:

- **Microsoft Reactor** : Free technical events
- **AI and Azure Meetups** : In-Person and Online
- **LinkedIn and GitHub** : Share projects and articles
- **Hackathons and bootcamps** : intensive practice with mentoring

12.7 Final Exercise

Objective : Create a career path in AI with a focus on Azure.

Steps :

1. Choosing a target certification
2. Define learning path in Microsoft Learn
3. Create a hands-on project with Azure ML
4. Post to GitHub and LinkedIn
5. Participate in a technical event

✅ 12.8 Career Checklist

- [x] Chosen certification
- [x] Study trails started
- [x] Practical project published
- [x] Updated portfolio
- [x] Networking started

🧠 Project: Image Classifier with Deep Learning and Web Interface

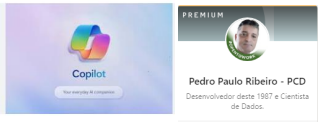
🎯 Objective

Create a web application that allows a user to upload an image and receive a classification based on a Deep Learning model trained on Azure.

📦 Project Structure

Code

```
image-classifier/
│
├── model/
│   └── model.pt # Trained model (PyTorch)
│
├── backend/
│   ├── app.py # Flask API
│   └── score.py # Inference logic
│
├── frontend/
│   └── app.py # Streamlit interface
│
├── requirements.txt # Dependencies
└── README.md # Documentation
```



Voluntary Contribution

https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

1. Model Training (Azure ML)

Treine um modelo CNN com PyTorch e salve como `model.pt`.

python

```python
# Exemplo simplificado de modelo
import torch.nn as nn

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(3, 16, 3), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3), nn.ReLU(), nn.MaxPool2d(2)
        )
        self.fc = nn.Sequential(
            nn.Linear(32 * 6 * 6, 128), nn.ReLU(),
            nn.Linear(128, 3)  # 3 classes
        )

    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)
        return self.fc(x)
```

2. Back-End com Flask

python

```python
# backend/app.py
from flask import Flask, request, jsonify
from score import predict_image

app = Flask(__name__)

@app.route('/predict', methods=['POST'])
def predict():
    file = request.files['image']
    result = predict_image(file)
    return jsonify({'class': result})

if __name__ == '__main__':
    app.run(debug=True)
```

python

```python
# backend/score.py
from PIL import Image
import torch
from torchvision import transforms
```

```
from model import CNN

model = CNN()
model.load_state_dict(torch.load('model/model.pt'))
model.eval()

def predict_image(file):
    image = Image.open(file).convert('RGB')
    transform = transforms.Compose([
        transforms.Resize((32, 32)),
        transforms.ToTensor()
    ])
    input_tensor = transform(image).unsqueeze(0)
    output = model(input_tensor)
    _, predicted = torch.max(output, 1)
    classes = ['Maçã', 'Banana', 'Laranja']
    return classes[predicted.item()]
```

3. Front-End com Streamlit

python

```
# frontend/app.py
import streamlit as st
import requests

st.title("Classificador de Frutas 🍎🍌🍊")
uploaded_file = st.file_uploader("Envie uma imagem", type=["jpg", "png"])

if uploaded_file:
    st.image(uploaded_file, caption="Imagem enviada", use_column_width=True)
    files = {'image': uploaded_file.getvalue()}
    response = requests.post("http://localhost:5000/predict", files=files)
    st.success(f"Classe prevista: {response.json()['class']}")
```

4. requirements.txt

txt

```
flask
torch
torchvision
Streamlit
pillow
requests
```

Voluntary Contribution
https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

5. Execution

1. Start the backend:

```bash
CD backend
python app.py
```

2. Start the frontend:

```bash
CD Frontend
streamlit run app.py
```

6. Possible Expansions

- Deploy the model to Azure as a REST API
- Use Azure Blob Storage to store images
- Add authentication with Azure Active Directory
- Monitor usage with Application Insights

Project Deployment on Azure

1. Prerequisites

- Azure Account
- Azure Machine Learning workspace created
- Saved trained model (e.g. `model.pt`)
- Inference script (`score.py`)
- Environment file (`env.yml` or `inference_config`)
- Installed Azure ML SDK (`pip install azureml-sdk`)

2. Register the Model

```python
from azureml.core import Workspace, Model

ws = Workspace.from_config()

model = Model.register(workspace=ws,
```

```
                        model_path="model/model.pt",
                        model_name="classificador_frutas")
```

3. Criar o Script de Inferência (score.py)

python

```
import torch
from torchvision import transforms
from PIL import Image
import json
import os

def init():
    global model
    model_path = os.path.join(os.getenv('AZUREML_MODEL_DIR'), 'model.pt')
    model = torch.load(model_path)
    model.eval()

def run(raw_data):
    try:
        data = json.loads(raw_data)
        image = Image.open(data['image']).convert('RGB')
        transform = transforms.Compose([
            transforms.Resize((32, 32)),
            transforms.ToTensor()
        ])
        input_tensor = transform(image).unsqueeze(0)
        output = model(input_tensor)
        _, predicted = torch.max(output, 1)
        classes = ['Maçã', 'Banana', 'Laranja']
        return classes[predicted.item()]
    except Exception as e:
        return str(e)
```

4. Definir o Ambiente

python

```
from azureml.core.environment import Environment
from azureml.core.model import InferenceConfig

env = Environment(name="inference-env")
env.python.conda_dependencies.add_pip_package("torch")
env.python.conda_dependencies.add_pip_package("torchvision")
env.python.conda_dependencies.add_pip_package("pillow")

inference_config = InferenceConfig(entry_script="score.py", environment=env)
```

5. Implantar com Azure Container Instances (ACI)

```python
from azureml.core.webservice import AciWebservice, Webservice

deployment_config = AciWebservice.deploy_configuration(cpu_cores=1, memory_gb=1)

service = Model.deploy(workspace=ws,
                       name="classificador-api",
                       models=[model],
                       inference_config=inference_config,
                       deployment_config=deployment_config)

service.wait_for_deployment(show_output=True)
print(service.scoring_uri)
```

6. Testar o Endpoint

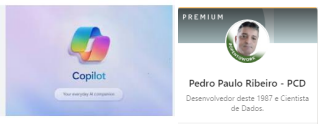
```python
import requests
import json

url = service.scoring_uri
headers = {'Content-Type': 'application/json'}

data = json.dumps({'image': 'caminho/para/imagem.jpg'})
response = requests.post(url, data=data, headers=headers)
print(response.json())
```

7. Monitoring and Logs

- Access Azure ML Studio
- Go to the deployed service
- View usage metrics, response time, and error logs
- Configure alerts with Application Insights



Voluntary Contribution

https://www.linkedin.com/in/pedro-paulo-ribeiro-pcd-1451498/
Whatsapp 55 21 999618643

Deep Learning on AZURE.

✅ Final Checklist

- [x] Registered model
- [x] Inference script created
- [x] Defined environment
- [x] Service implemented with ACI
- [x] Endpoint tested successfully
- [x] Monitoring enabled

🚀 Integration and Deployment of a Deep Learning Project on Azure

📦 1. Project Structure

Code

```
image-classifier/
├── model/ # Trained model (.pt, .pkl, .onnx)
├── score.py # Inference script
├── inference_config.py # Runtime environment
├── app.py # Web interface (Flask or Streamlit)
├── azure_deploy.py # Azure deployment script
└── requirements.txt # Dependencies
```

🧠 2. Registering the Model on Azure

Python

```
from azureml.core import Workspace, Model

ws = Workspace.from_config()

model = Model.register(workspace=ws,
model_path="model/model.pt",
model_name="fruit_classifier")
```

⚙️ 3. Script de Inferência (`score.py`)

python

```
import torch
from torchvision import transforms
from PIL import Image
```

```
import json
import os

def init():
    global model
    model_path = os.path.join(os.getenv('AZUREML_MODEL_DIR'), 'model.pt')
    model = torch.load(model_path)
    model.eval()

def run(raw_data):
    try:
        image = Image.open(raw_data['image']).convert('RGB')
        transform = transforms.Compose([
            transforms.Resize((32, 32)),
            transforms.ToTensor()
        ])
        input_tensor = transform(image).unsqueeze(0)
        output = model(input_tensor)
        _, predicted = torch.max(output, 1)
        classes = ['Maçã', 'Banana', 'Laranja']
        return classes[predicted.item()]
    except Exception as e:
        return str(e)
```

4. Definição do Ambiente

python

```
from azureml.core.environment import Environment
from azureml.core.model import InferenceConfig

env = Environment(name="inference-env")
env.python.conda_dependencies.add_pip_package("torch")
env.python.conda_dependencies.add_pip_package("torchvision")
env.python.conda_dependencies.add_pip_package("pillow")

inference_config = InferenceConfig(entry_script="score.py", environment=env)
```

5. Implantação com Azure Container Instances (ACI)

python

```
from azureml.core.webservice import AciWebservice, Webservice

deployment_config = AciWebservice.deploy_configuration(cpu_cores=1, memory_gb=1)

service = Model.deploy(workspace=ws,
```

```
                        name="classificador-api",
                        models=[model],
                        inference_config=inference_config,
                        deployment_config=deployment_config)

service.wait_for_deployment(show_output=True)
print("Endpoint:", service.scoring_uri)
```

6. Integração com Aplicação Web (Flask ou Streamlit)

Exemplo com Streamlit:

python

```
import streamlit as st
import requests

st.title("Classificador de Frutas 🍎🍌🍊")
uploaded_file = st.file_uploader("Envie uma imagem", type=["jpg", "png"])

if uploaded_file:
    st.image(uploaded_file, caption="Imagem enviada", use_column_width=True)
    files = {'image': uploaded_file.getvalue()}
response = requests.post("https://<endpoint>", files=files)
st.success(f"Predicted class: {response.json()['class']}")
```

7. Automation with Azure Functions (Optional)

- Create an Azure Function that calls the template endpoint
- Integrate with Logic Apps to trigger automatic flows
- Use Power Automate to send alerts or save results

8. Monitoring and Logs

- Use **Application Insights** to track calls, errors, and response times
- Configure alerts with **Azure Monitor**
- Enable **Model Drift Detection** to track performance over time

Final Checklist

Stage	Status
Trained and saved model	✅
Inference script created	✅
Defined environment	✅
Registered model	✅
Endpoint deployed	✅
Connected application	✅
Monitoring activated	✅